



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Laboratorio di Sicurezza Informatica

Sicurezza fisica e collocazione delle risorse

Marco Prandini

Dipartimento di Informatica – Scienza e Ingegneria

On premises vs. in the cloud

- Abbiamo visto sistemi di controllo dell'accesso alle informazioni; ricordiamo che sono tutti “mediati”
 - dal sistema operativo (es. permessi su file)
 - dalla gestione di segreti (es. chiavi crittografiche)
- Come influisce la collocazione delle risorse?
- On premises:
 - nessuna condivisione
 - relativa semplicità di separazione tra segreti e sistemi e dati
 - inserimento manuale di password e chiavi
 - necessità di garantire
 - il corretto funzionamento dell'hardware
 - l'esecuzione di sistemi operativi e software integri e autentici
- In the cloud
 - gestione hw/sw di altissimo livello
 - ambienti di memorizzazione ed elaborazione condivisi
 - delocalizzazione

On premises - messa in sicurezza fisica

- Un server è prima di tutto un sistema di calcolo, collocato in un ambiente e connesso a una varietà di dispositivi
 - Normalmente si concentrano le difese sul fronte degli attacchi via rete, a componenti software come applicazioni e sistema operativo
 - Le corrispondenti contromisure possono facilmente essere scavalcate da un attaccante con accesso fisico al sistema!
 - Le minacce principali sono:
 - Furto dello storage o dell'intero calcolatore
 - Connessione di sistemi di raccolta dati alle interfacce
 - Avvio del sistema con un sistema operativo arbitrario
 - La gravità di queste minacce dipende fortemente dallo specifico ambiente
- Molti di questi problemi sono cambiati nello scenario sempre più comune di virtualizzazione sul Cloud, ma altri concettualmente simili sono apparsi, e la logica delle stesse contromisure si può adattare

Fattori non informatici

- Vedremo che l'accesso a sistema permette attacchi specifici
- Come si ottiene l'accesso?
 - Insider
 - Tailgating
 - Errata identificazione di visitatori
 - Social engineering
 - Effrazione
- La sicurezza fisica “tradizionale” è essenziale!
 - Regolamenti chiari e condivisi
 - Perimetro robusto
 - Sorveglianza e procedure

<https://docs.microsoft.com/it-it/azure/security/fundamentals/physical-security>
- La disponibilità è il terzo vertice della triade CIA
 - Alimentazione
 - Connettività
 - Condizionamento
 - Incidenti, disastri, attentati <https://goo.gl/maps/5Ukzcorg5pZNmbzW7>

Alcune vulnerabilità sfruttabili in presenza

- BadUSB e simili

<https://threatpost.com/badusb-attack-code-publicly-disclosed/108663/>

- Thunderspy

<https://thunderspy.io/>

- Keylogging e videoghosting

<https://www.keelog.com/>

- Key injection

<https://www.blackhillsinfosec.com/executing-keyboard-injection-attacks/>

- Disk un/plugging

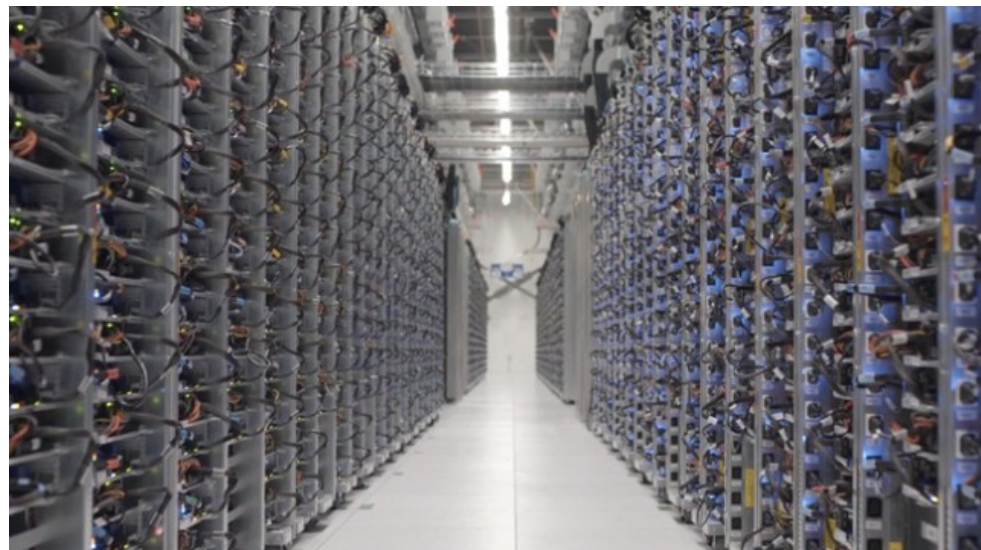
<https://www.blackhat.com/docs/eu-15/materials/eu-15-Boteanu-Bypassing-Self-Encrypting-Drives-SED-In-Enterprise-Environments-wp.pdf>

- Power glitching

<https://www.darkreading.com/edge/theedge/glitching-the-hardware-attack-that-can-disrupt-secure-software-/b/d-id/1336119>



In remoto



- Se la collocazione è fuori dalla possibilità di controllo diretto, si può considerare di:
 - Scegliere un case che possa essere chiuso e fissato al rack
 - Installare dispositivi di rilevazione delle intrusioni
 - Adottare misure di protezione dei dati che rendano inutile il furto
 - L'accesso ai dati va però abilitato manualmente
 - Disabilitare le periferiche non utilizzate
 - Salvo poi averne bisogno per esigenze nuove

Attacchi fisici alle risorse logiche

- Per andare a regime il sistema attraversa un processo di boot, che può essere diviso in queste fasi:
 - (1) BIOS – Individua i dispositivi di possibile caricamento del boot loader e l'ordine per esaminarli
 - Molti BIOS prevedono la possibilità di proteggere con password l'avvio o la modifica della configurazione
 - (2) Boot Loader – Sceglie il sistema operativo e gli passa eventuali parametri
 - Gestione della “maintenance mode”
 - Stesso tipo di protezione con password come descritto per BIOS
 - (3) Sistema operativo – carica i device driver (da non sottovalutare) e avvia il processo *init*
 - (4) *init* – gestisce i *runlevel* o i *target* per coordinare l'inizializzazione del sistema, cioè avviare i servizi nell'ordine corretto
- Ognuna di queste fasi potrebbe essere **dirottata** da un attaccante con accesso fisico, per far caricare software malevolo

Boot

■ Password pro e contro:

- Se serve una password per l'avvio del sistema, il funzionamento non supervisionato può essere problematico: ad esempio può non ripartire per ore dopo una semplice mancanza di alimentazione
 - In sistemi con gravi esigenze di sicurezza, non sarà comunque l'unica password da fornire, quindi meglio proteggere tutti gli strati
- Almeno la protezione contro i cambi di configurazione è sempre consigliabile

■ MAI affidarsi a un unico strato di protezione

- Le password del BIOS hanno meccanismi semplici di reset
- Possono essere indovinate...

Bootloader – configurazione (runtime)

- LILO, the Linux Loader
 - Usato fin dagli albori di Linux
- GRUB, the Grand Unified Bootloader
 - GRUB è più potente e flessibile di LILO, è dotato di una shell che permette di eseguire vari comandi per modificare al volo la procedura di avvio: naturalmente questo permette molti abusi
- Entrambi permettono di passare parametri al kernel, i più importanti ai fini della sicurezza sono
 - single
 - Alcune distribuzioni hanno default rischiosi, ad esempio se si riesce a innescare un *maintenance mode* aprono una shell di root senza chiedere password
 - init=...
 - Permette di sostituire il comando di default lanciato dal kernel come primo processo del sistema (normalmente /sbin/init)
 - E se fosse /bin/bash?

Boot Loader passwords

■ LILO

`password=YourPasswordHere`

Imposta una password richiesta al boot, a meno che

`restricted`

non sia specificato, in tal caso la password è richiesta solo per modificare i parametri durante il boot.

■ Global vs. Single-entry

- `password` e `restricted` nella “global section”: chiede la password prima di consentire l’aggiunta di parametri – attenzione alle entry non sicure (cd)
- `password` e `restricted` in una “image section”: chiede la password prima di consentire l’aggiunta di parametri, solo per l’immagine specificata
- `password` nella “global section” e `restricted` in una “image section”: chiede la password prima di consentire l’aggiunta di parametri, solo per l’immagine specificata, mentre chiede sempre la password per avviare altre immagini

Boot Loader passwords

■ GRUB

`password [--md5] passwd [new-config-file]`

Se specificato nella “global section”, imposta una password che sarà richiesta per attivare l'*interactive operation* del bootloader. Opzionalmente può innescare il caricamento di un file di configurazione alternativo

Se specificata per un item specifico del menu, imposta una password che sarà richiesta per avviare quell'item

`lock`

Se specificata per un item specifico del menu, subito dopo `title`, contrassegna quell'item come password-protected.

Funziona solo se esiste una direttiva `password` nella “global section”

`md5crypt`

Comando utilizzabile al grub prompt per calcolare il password hash da usare con `--md5`

Sicurezza del processo di boot

- Problema: come assicurarsi che ogni componente software eseguito da un computer sia autentico, integro e benevolo?
 - Anti-malware verificano le applicazioni
 - Chi verifica gli anti-malware?? Il S.O. (idealmente rendendo AM inutile)
 - Chi verifica il S.O.? Il boot loader potrebbe
 - Chi verifica il boot loader? Il BIOS potrebbe, specialmente se assistito da HW speciale, che non possa essere modificato dal S.O., e quindi sia immune da infezioni
- *hardware root of (a chain of) trust*

https://medium.com/@martin_24447/trusted-boot-b1ae7e6d2890

<https://dl.acm.org/doi/pdf/10.1145/3380774.3382016>

Measured / Trusted / Secure Boot

- **Measured Boot** si riferisce a un processo generale, che tipicamente usa un **TPM** come hardware root of trust
 - TPM = Trusted Platform Module: chip con funzionalità crittografiche
 - fa parte delle specifiche del *Trusted Computing Group*
<https://trustedcomputinggroup.org/>
 - M.B. non definisce come **prevenire** un avvio malevolo
- **Trusted Boot** è un processo che usa gli strumenti del M. B. e riesce a bloccare il boot non appena individua un componente non fidato
- **Secure Boot** è il nome specifico dato all'implementazione di trusted boot basata su **UEFI**
 - UEFI = Unified Extensible Firmware Interface
<http://www.uefi.org/>
 - Implementazione Software + chiavi in firmware
 - Serve un BIOS standard per la fase di POST
 - Può avvalersi del TPM per velocizzare e migliorare i controlli di integrità

Measured boot

■ Basato sul TPM

- Core Root of Trust for Measurement (**CRTM**)
- Registers (**PCR**)

■ Raccoglie hash di ogni componente caricato

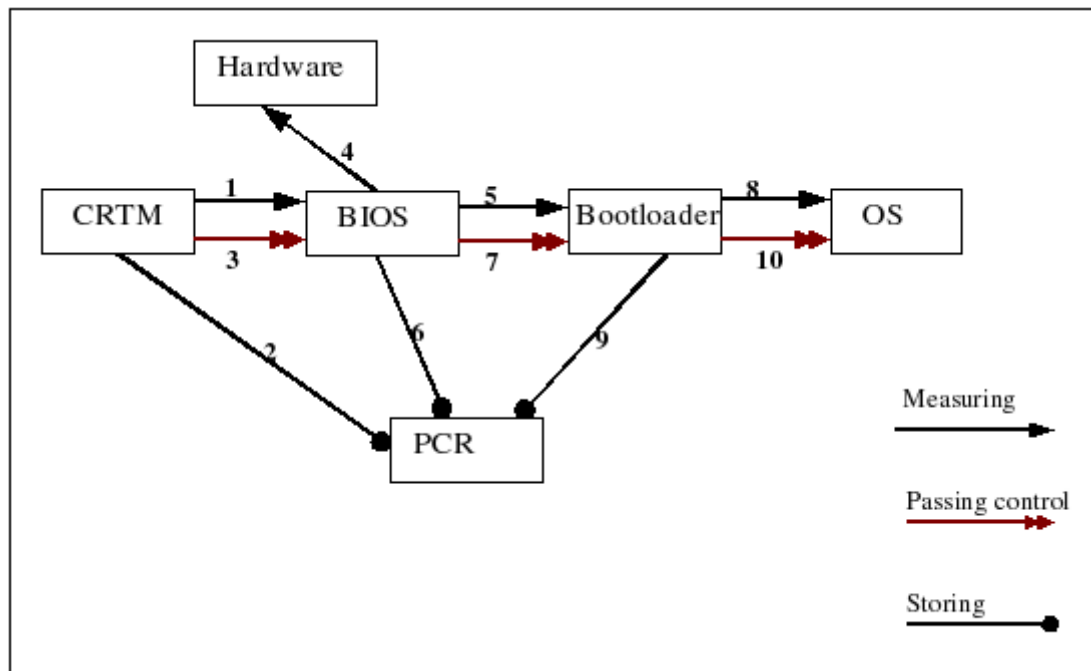
- nei PCR che sono fisicamente non modificabili una volta scritti

■ Pospone i controlli fintanto che non dispone

- Delle crypto keys
- Di abbastanza memoria per fare i calcoli necessari

■ Si può decidere chi fa i controlli e quando

- per esempio dall'esterno (sistema fidato) per abilitare funzioni critiche
- **remote attestation!**



UEFI e secure boot

- EFI (Intel) nasce come interfaccia più flessibile del BIOS tra S.O. e firmware
- UEFI forum standardizza e aggiorna la specifica
- UEFI è un “mini OS”
 - milioni di righe di codice
 - standard per molte piattaforme
 - bersaglio ideale degli attaccanti!
https://media.kaspersky.com/en/business-security/Threats_to_UEFI.pdf
<https://www.csoonline.com/article/3599908/trickbot-gets-new-uefi-attack-capability-that-makes-recovery-incredibly-hard.html>
<https://www.debian.org/security/2021-GRUB-UEFI-SecureBoot/>
- UEFI verifica ogni componente software prima di passare il controllo a BootLoader/SistemaOperativo
 - Richiede la disponibilità di un database di chiavi
 - Blocca il boot appena rileva una difformità

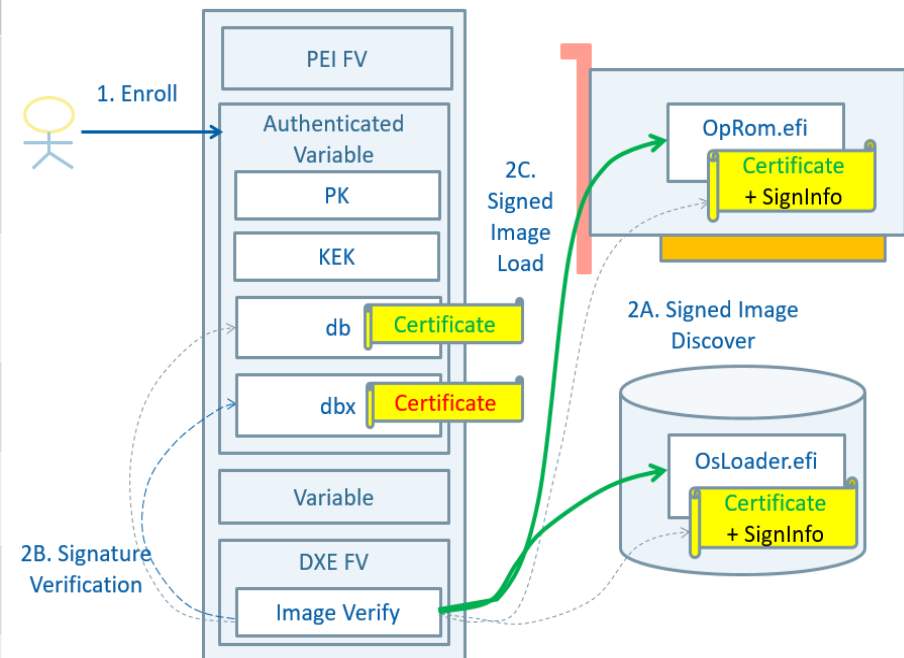
Le chiavi di UEFI Secure Boot

https://edk2-docs.gitbook.io/understanding-the-uefi-secure-boot-chain/secure_boot_chain_in_uefi/uefi_secure_boot

- UEFI Secure Boot definisce due processi di sicurezza:
 - verifica dell'immagine di boot
 - verifica degli aggiornamenti al database della sicurezza delle immagini
- Per fare questo si avvale di differenti database e set di chiavi

Key	Verifies	Update is verified by	NOTES
PK	New PK New KEK New db/dbx/dbt/dbr New OsRecoveryOrder New OsRecovery####	PK	Platform Key
KEK	New db/dbx/dbt/dbr New OsRecoveryOrder New OsRecovery####	PK	Key Exchange Key
db	UEFI Image	PK/KEK	Authorized Image Database
dbx	UEFI Image	PK/KEK	Forbidden Image Database
dbt	UEFI Image + dbx	PK/KEK	Timestamp Database
dbr	New OsRecoveryOrder New OsRecovery####	PK/KEK	Recovery Database

UEFI Secure Boot



UEFI Secure Boot Image Verification

https://edk2-docs.gitbook.io/understanding-the-uefi-secure-boot-chain/secure_boot_chain_in_uefi/uefi_secure_boot

■ Entità coinvolte nel processo di verifica delle immagini al boot

- TP = trusted platform, procedura di verifica
- CDI = UEFI Secure Boot Image Security Database
- UDI = qualsiasi firmware di terze parti, inclusi boot loader, PCI option ROMs, o UEFI shell tool.

■ Al boot, TP verifica l'integrità di UDI utilizzando le policy CDI

- se ok, UDI entra a far parte di CDI e il firmware di terze parti viene eseguito

■ Il CDI, cioè il database delle politiche di sicurezza da applicare alle immagini software da caricare, è quindi aggiornabile.

- Il fornitore del componente deve firmarlo con la propria chiave privata e rendere disponibile la chiave pubblica.
- la chiave pubblica deve essere iscritta (enrolled) nel firmware del sistema
- normalmente questo passaggio richiede un reboot in una modalità speciale e l'intervento sulla console, bloccando quindi l'azione di utenti malevoli ma senza accesso fisico al sistema

Item	Entity	Provider	Location
TP	UEFI Secure Boot Image Verification	OEM	Originally on flash, loaded into DRAM
CDI	Manufacture Firmware Code	OEM	Originally on flash, loaded into DRAM
	UEFI Secure Boot Image Security Database (Policy)	End user (or OEM default)	Originally on flash, authenticated variable region, loaded into DRAM
UDI	3rd party Firmware Code, (OS boot loader)	OSV	Originally on external storage (e.g. Hard drive, USB), loaded into DRAM
	3rd party Firmware Code, (PCI Option ROM)	IHV	Originally on PCI card, loaded into DRAM
	3rd party Firmware Code, (UEFI Shell Tool)	Any	External Storage (e.g. hard drive, USB), loaded into DRAM

UEFI e secure boot in Linux

https://www.suse.com/media/presentation/uefi_secure_boot_webinar.pdf
<https://wiki.ubuntu.com/UEFI/SecureBoot>

1) La Platform Key ufficiale verifica un piccolo pre-boot-loader, **shim**

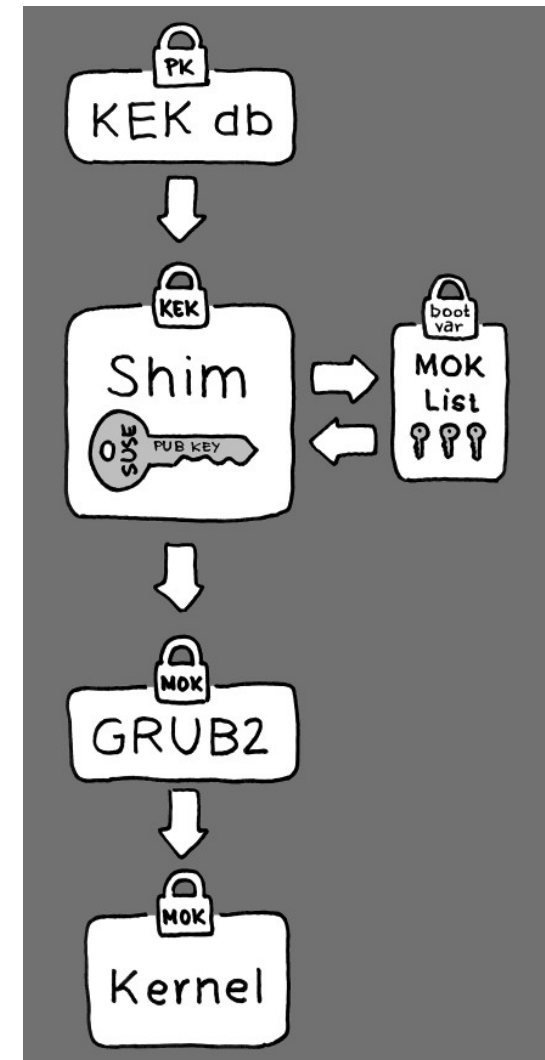
- La chiave key usata per “firmare” shim deve essere fornita dal costruttore HW
- È una chiave Microsoft!

2) Shim può usare o trasferire MOKs (Machine Owner Keys)

- Per validare il bootloader
- Per validare moduli custom del kernel

■ Componenti aggiuntivi del kernel devono essere firmati per poterli caricare

- L'utente genera le MOKs
- L'utente deposita le MOKs in shim
- Al boot successivo, shim trova le chiavi nella fase di setup, e chiede conferma per salvarle in firmware → **consenso esplicito e basato su password sempre richiesto!**



<https://www.suse.com/communities/blog/uefi-secure-boot-details/>

UEFI / Secure boot links

- E-book con schemi chiari e riferimenti a proprietà formali di sicurezza secondo il modello di Clark-Wilson garantite dal procedimento
 - <https://edk2-docs.gitbook.io/understanding-the-uefi-secure-boot-chain/>
- Documentazione originale
 - https://uefi.org/sites/default/files/resources/UEFI_Secure_Boot_in_Modern_Computer_Security_Solutions_2013.pdf
- Articoli datati ma illustrano i problemi incontrati all'introduzione di UEFI
 - <http://www.rodsbooks.com/linux-uefi/>
 - <http://www.linux-magazine.com/Online/Features/Coping-with-the-UEFI-Boot-Process>
 - <https://help.ubuntu.com/community/UEFI>
 - <http://askubuntu.com/questions/760671/could-not-load-vboxdrv-after-upgrade-to-ubuntu-16-04-and-i-want-to-keep-secure>
 - <https://www.suse.com/communities/blog/uefi-secure-boot-details/>
 - <https://lwn.net/Articles/519618/>
- Guide alla personalizzazione
 - <https://media.defense.gov/2020/Sep/15/2002497594/-1/-1/0/CTR-UEFI-Secure-Boot-Customization-UOO168873-20.PDF>
 - <https://www.linuxjournal.com/content/take-control-your-pc-uefi-secure-boot>

Integrità del software

- Il processo di secure boot garantisce l'integrità fino all'avvio del sistema operativo
- I pacchetti software applicativi sono meno “potenti” ma non necessariamente meno pericolosi

Autenticazione

- La prima cautela da usare quando si scarica software dovrebbe essere quella di verificarne l'autenticità da una firma digitale
- Naturalmente per verificare una firma serve una chiave pubblica fidata
- Es:
 - 1) `gpg --verify FILE.asc FILE.tar.gz`
 - (mostra il key id)
 - 2) `gpg --keyserver pgpkeys.mit.edu --recv-key <KEY_ID>`
 - l'autenticità della chiave in questo caso deriva solo dalla fonte... basta?
 - valutare caso per caso
 - seguire indicazioni specifiche <https://httpd.apache.org/dev/verification.html>
 - 3) ripetere il passo (1)
- Come minimo, dovrebbe essere disponibile un fingerprint
- Basta mettere il fingerprint (es. in formato .sha256) nella stessa directory del file da verificare e lanciare
 - `sha256 -c FILE.sha256`
- esempio di fonte che li offre entrambi
 - <https://httpd.apache.org/download.cgi>

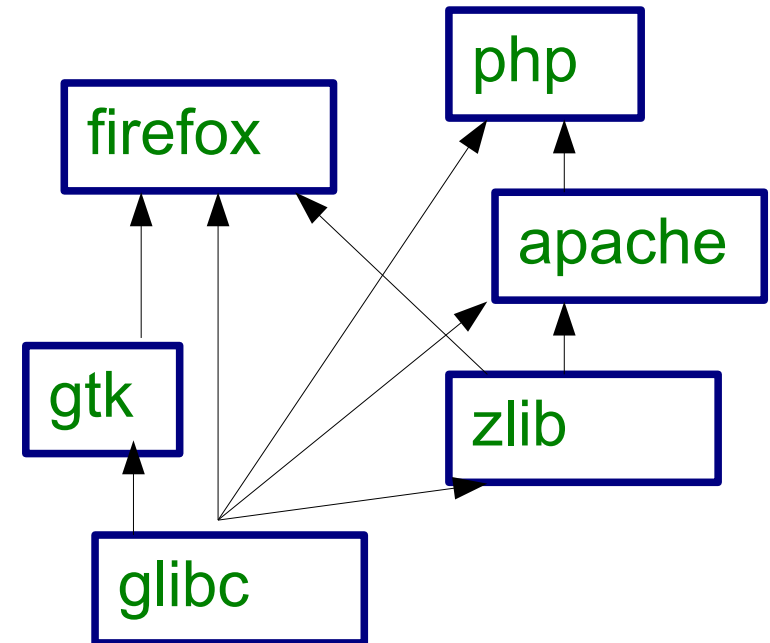
Riflessioni sull'installazione da sorgenti

- **Offre la possibilità teorica di verificare il codice.**
 - Se non lo fate → falso senso di sicurezza.
 - Se vi fidate della firma sull'archivio, non è diverso che verificare la firma su di un binario.
- **Più difficile da mantenere**
- **Richiede MOLTI componenti ausiliari**
 - Header e librerie
 - Processori di macro
 - Compilatori e linker ...
 - sono tutti elementi che possono avvantaggiare un attaccante
<https://wiki.c2.com/?TheKenThompsonHack>
- **Entrano in scena le distribuzioni e i pacchetti:**
 - Chiavi di verifica installate una volta per tutte
 - comodo ma SPOF
 - Gestione automatica delle dipendenze
 - Garanzia di compatibilità binaria tra tutti gli elementi del set

Installazione assistita

- Comunemente effettuata per mezzo di software ausiliari
 - package manager specifico della distribuzione Linux (rpm/yum, dpkg/apt, ...)
 - installer per Windows
- Un tool di installazione
 - può farsi carico delle verifiche relative alle dipendenze
 - non può configurare ogni dettaglio del sistema in modo specifico
 - può generare dinamicamente dati specifici

Esempio di *grafo delle dipendenze*:



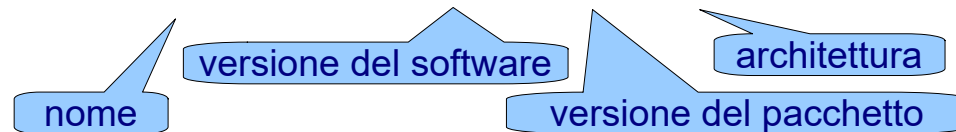
$A \rightarrow B$ significa che A “serve” per B; “serve” può essere una dipendenza tra funzionalità logiche (non ha senso avere un linguaggio di generazione pagine web senza un web server) o fisiche (un binario linkato dinamicamente non gira senza tutte le librerie di cui importa i simboli)

Debian e Red Hat

- Due distribuzioni capostipite da cui sono state derivate quasi tutte le varianti più diffuse
https://en.wikipedia.org/wiki/List_of_Linux_distributions
-
- Due sistemi di gestione dei pacchetti con molte somiglianze
 - Tool di basso livello per la gestione dei singoli pacchetti
 - Tool intermedi per la gestione coordinata di pacchetti e dipendenze
 - Tool per il reperimento automatico da *repository* dei pacchetti necessari

Pacchetti

- I pacchetti per le distribuzioni Debian e derivate (es. Ubuntu) sono in formato *.deb*
 - **aptitude**-0.2.15.9-2_**i386**.deb



- I pacchetti per le distribuzioni RedHat e derivate (es. CentOS, Fedora) sono in formato *.rpm*
 - **httpd**-2.4.6-45.el7.centos.**x86_64**.rpm

Repository

- I pacchetti possono essere scaricati e gestiti singolarmente
- Normalmente però si usano i repository (repo)
 - raccolte indicizzate di pacchetti
 - possono essere online o su filesystem locali
- I **package manager** leggono per ogni repo l'indice e i metadati dei pacchetti
 - conoscono quali versioni sono disponibili per ogni pacchetto
 - conoscono le dipendenze tra pacchetti (e quindi come risolverle)
- Collocazioni delle liste di repo ed esempi:
 - `/etc/apt/sources.list` `/etc/apt/sources.list.d/*`
`deb http://archive.ubuntu.com/ubuntu bionic-updates universe`
 - `/etc/yum.conf` `/etc/yum.repos.d/*.repo`
`[base]`
`name=CentOS-$releasever - Base`
`mirrorlist=http://mirrorlist.centos.org/?`
`release=$releasever&arch=$basearch&repo=os&infra=$infra`
`#baseurl=http://mirror.centos.org/centos/$releasever/os/$basearch/`
`gpgcheck=1`
`gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-CentOS-7`

Verifica dell'autenticità

- La firma dei pacchetti è gestita centralmente
- I *maintainer* di una distribuzione forniscono le chiavi di verifica nei media di installazione ufficiali o sui *repository* online
- I set di chiavi possono essere gestiti in modo standard con GnuPG
es. i sistemi .deb-based mettono gpg keyrings in `/etc/apt/trusted.gpg.d/`
- ... ma è più comune usare strumenti forniti dalla distribuzione

.deb `apt-key {add file | list | del keyid | adv --recv-key keyid | ... }`

.rpm `rpm {--import | -e | -q[ai] | ...}`

Deprecato da molto tempo ma ancora in uso
Mette tutte le chiavi nello stesso file
`/etc/apt/trusted.gpg`

rpm tratta le chiavi come se fossero pacchetti
→ si possono usare gli stessi
comandi per interrogarli, eliminarli, ecc.

Lavorare coi repository

- Un'esigenza molto comune è quella di installare software ben supportato ma non incluso per qualsiasi motivo nei canali ufficiali della distribuzione
- Si aggiunge semplicemente il repository all'elenco *succede che non tutti supportino l'autenticazione* :-(
 - Apt (deb):

```
/etc/apt/sources.list.d/trusted.list :  
deb [allow-insecure=yes] http://good.site/virtualbox/debian bookworm contrib
```
 - Yum (rpm):

```
/etc/yum.repos.d/epel.repo :  
[epel]  
name=Epel Linux -  
baseurl=http://mirror.example.com/repo/epel5_x86_64  
enabled=1  
gpgcheck=0
```

Cross-signing threat

- In distro deb-based era diffuso l'uso di righe di configurazione prive di specifica della chiave

```
deb http://download.virtualbox.org/virtualbox/debian xenial contrib
```
- In questo caso vengono usate tutte le chiavi collocate in posti "fidati"
 - `/etc/apt/trusted.gpg`
 - `/etc/apt/trusted.gpg.d/*`
- Una chiave importata per convalidare una fonte di pacchetti può essere usata per tutti → **cross signing**
 - se un repo viene violato e la chiave privata sottratta, può pubblicare aggiornamenti fraudolenti per tutti i pacchetti anche degli altri
- Best practices correnti:
 - Opzioni sulla singola riga del file di configurazione

```
deb [arch=amd64 signed-by=/usr/share/keyrings/oracle-virtualbox-2016.gpg]  
https://download.virtualbox.org/virtualbox/debian bookworm contrib
```
 - File di configurazione con struttura simile al mondo rpm

```
/etc/apt/sources.list.d/ubuntu.sources  
Types: deb  
URIs: http://it.archive.ubuntu.com/ubuntu/  
Suites: resolute resolute-updates resolute-backports  
Components: main restricted universe multiverse  
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

Gestire la provenienza dei pacchetti

- Si può generare confusione se un pacchetto con lo stesso nome è presente in versioni diverse in repository differenti
 - I package manager, di default, scelgono sempre la versione più avanzata
 - supponiamo di aver aggiunto un repo semisconosciuto per installare un'applicazione innocua
 - se a tale repo viene aggiunto un pacchetto "core" dichiarato più recente della versione ufficiale → **software injection!**
- In alcuni casi anche aggiornamenti nello stesso repo sono indesiderabili
 - situazioni legacy
- La situazione va controllata e gestita
 - Controllo della provenienza di un pacchetto
 - Yum: `repoquery -i [package name]`
 - Apt: `apt-cache showpkg [package name]`
 - Elenco dei pacchetti provenienti da un repo
 - Yum: `yum list installed | grep [repo name]`
 - Apt: vari comandi per estrarre manualmente info dai file della cache

Limitare le modifiche automatiche

- Per evitare a priori problemi in sistemi con dipendenze complesse (ad esempio mix di pacchetti installati manualmente e via package manager)
 - Version locking/pinning
 - Apt
 - editare `/etc/apt/preferences.d/*`
 - <https://wiki.debian.org/AptPreferences>
 - Yum
 - `yum install yum-plugin-versionlock`
 - poi
 - `yum versionlock [package name]`
 - o editare a mano
 - `/etc/yum/pluginconf.d/versionlock.list`

DevOps woes

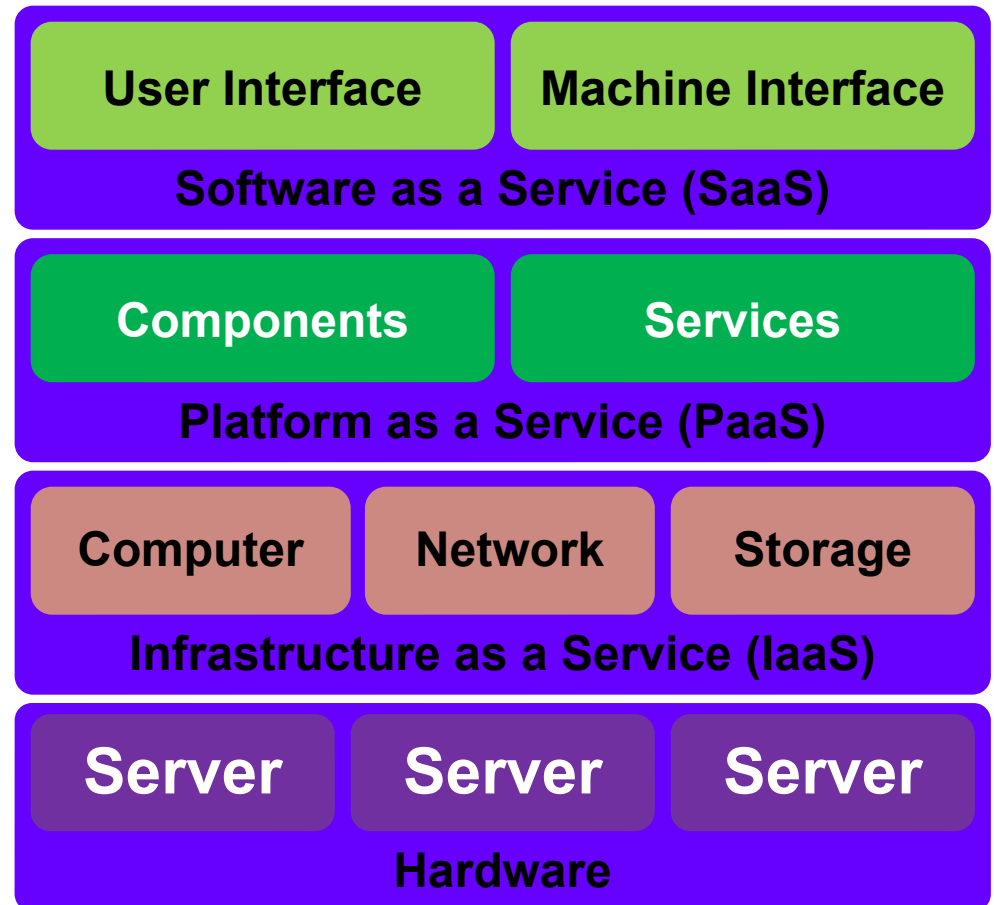
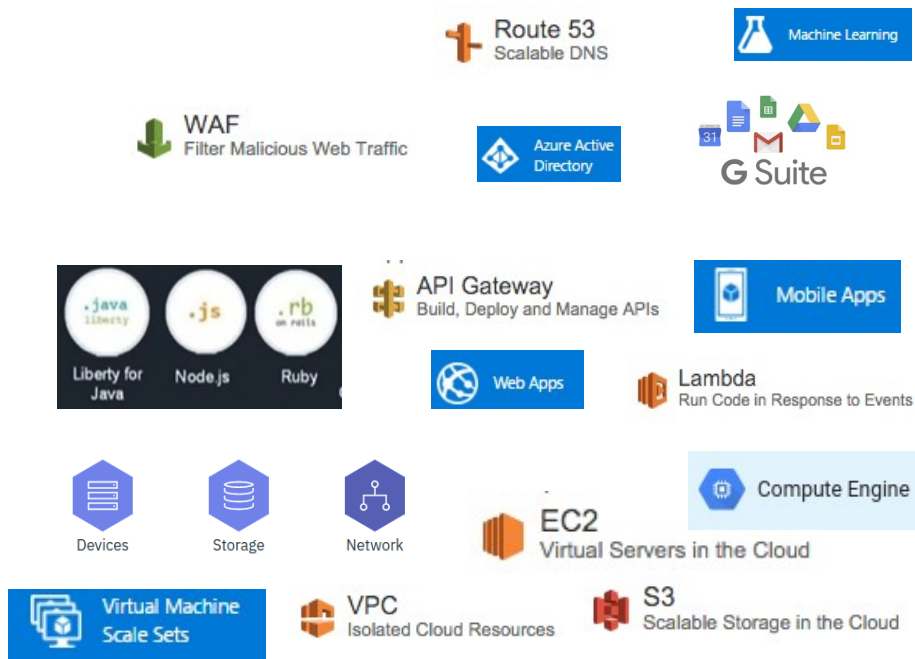
- DevOps: development + operations
 - automatizzare il build e deployment del software
 - **inclusione automatica delle dipendenze**
- Raramente c'è una gestione corretta dei **namespace**
 - es. *npm* per l'ambiente *node* → praticamente flat
 - supponiamo che un'azienda sviluppi internamente un pacchetto “myutils”
 - un attaccante registra un pacchetto “myutils” nel repo di default
 - al prossimo build, il sistema di CI/CD quale prende?

<https://www.sonatype.com/blog/dependency-hijacking-software-supply-chain-attack-hits-more-than-35-organizations>

<https://www.sonarsource.com/blog/securing-developer-tools-a-new-supply-chain-attack-on-php/>

Due parole su cloud security

- A seconda del livello utilizzato, molti problemi “svaniscono”



from Principles, Applications and Models for Distributed Systems M – Antonio Corradi & Luca Foschini

Sicurezza: cloud vs. on-premise

- I problemi di sicurezza del cloud riguardano raramente la sicurezza dell'host e della rete
- Impatto "emotivo" della distanza
 - perdita di controllo
- Osservazione razionale
 - il più delle volte, i fornitori di cloud hanno team di sicurezza di livello mondiale che quasi nessuna azienda può permettersi di assumere per i propri data center
- la due diligence suggerisce di verificare questa affermazione quando si seleziona un fornitore di servizi cloud!
 - Certificazione ISO27001 + ambito di applicazione
 - risultati di un audit SAS70 di tipo II
- Paradossalmente, alcune minacce reali riguardano la disponibilità
 - lock-in
 - dipendenza dalla rete
 - cessazione dell'attività

I benefici potenziali del cloud per la sicurezza

- **Le misure di sicurezza sono più economiche se implementate su scala più ampia:**
 - Collocazioni multiple = ridondanza di istanze e dati = indipendenza dai guasti dovuti a comportamenti dannosi, opportunità di ripristino
 - Tempestività migliorata: è possibile utilizzare sistemi su scala più ampia per sviluppare capacità di risposta agli incidenti più efficaci
 - Gestione delle minacce: i fornitori di servizi cloud possono coinvolgere specialisti nella gestione di minacce alla sicurezza specifiche
- **La sicurezza come elemento di differenziazione sul mercato**
 - Il cliente del servizio cloud (CSC) effettuerà le scelte di acquisto di servizi e risorse sulla base della reputazione di riservatezza, integrità e resilienza del fornitore di servizi cloud (CSP), nonché dei servizi di sicurezza offerti da CSP
 - Questo è un forte motore per i CSP per migliorare le pratiche di sicurezza

I benefici potenziali del cloud per la sicurezza

■ Aggiornamenti più tempestivi ed efficaci

- Le immagini delle macchine virtuali possono essere pre-rafforzate e aggiornate con le ultime patch e impostazioni di sicurezza in modo centralizzato, riducendo al minimo le vulnerabilità
- IaaS può fornire un servizio per consentire l'acquisizione regolare di istantanee dell'infrastruttura virtuale, nonché la rapida implementazione di numerosi aggiornamenti su piattaforme omogenee

■ Concentrazione delle risorse e rapido ridimensionamento

- La concentrazione delle risorse consente l'applicazione più semplice ed economica di controlli di sicurezza completi e policy sui processi di manutenzione, gestione degli incidenti e gestione delle patch
- CSP può scalare dinamicamente i meccanismi difensivi su richiesta per il traffic shaping, il filtraggio, la crittografia, ecc., al fine di aumentare il supporto per le misure di riparazione durante un attacco
- Quando queste capacità sono combinate con un appropriato schema di ottimizzazione delle risorse, il CSP può limitare l'effetto che alcuni attacchi potrebbero avere sulla disponibilità delle risorse che ospitano i servizi cloud, nonché limitare l'uso delle risorse necessarie per affrontare tali attacchi

I benefici potenziali del cloud per la sicurezza

■ Sicurezza come servizio e raccolta di prove

- I CSP di grandi dimensioni possono offrire come servizio interfacce standardizzate per la sicurezza gestita; questo potenzialmente crea un mercato per i servizi di sicurezza, dove i CSC possono facilmente acquisire controlli di sicurezza preconfigurati con costi di set-up inferiori
- IaaS può offrire supporto per la sicurezza su richiesta, clonare macchine virtuali (VM) e fornire uno storage conveniente per i log, che può essere utilizzato per analisi forensi offline, senza compromettere le prestazioni

■ Gli SLA impongono una migliore gestione del rischio

- I CSP implementano procedure di audit interno e valutazione del rischio più rigorose per quantificare le sanzioni per gli scenari di rischio negli SLA e definire azioni correttive per ridurre il possibile impatto delle violazioni della sicurezza sulla propria reputazione

Attori e minacce nel cloud

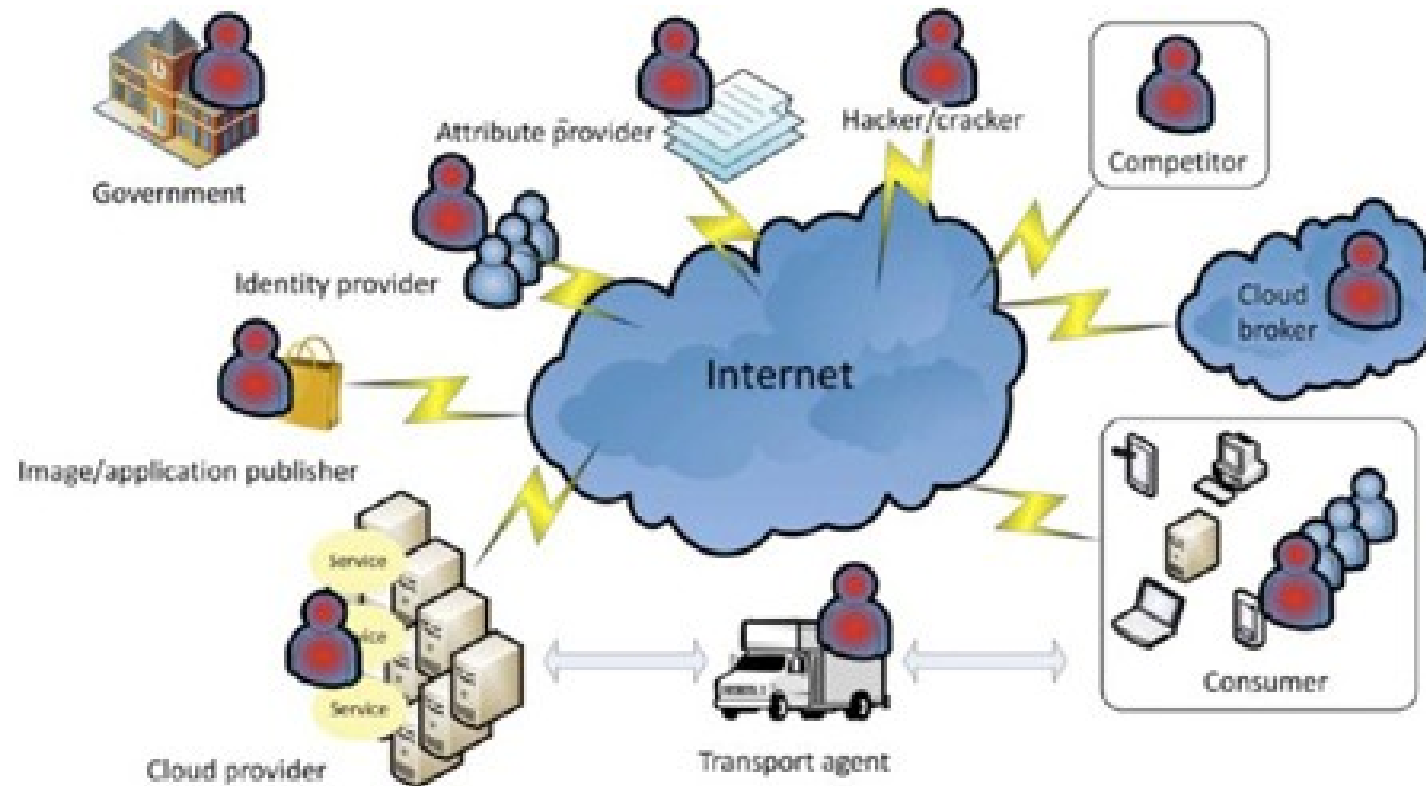


Image from "Securing Cloud Services: A pragmatic approach to security architecture in the Cloud"
by Lee Newcombe - IT Governance Publishing

I rischi specifici del cloud

- Tre categorie principali
- Rischi Organizzativi
 - Perdita di controllo
 - Lock-in
 - Supply Chain Failure
 -
- Rischi Tecnici
 - Economic denial of service (EDoS)
 - Cedimento dell'isolamento
 -
- Rischi legali
 - Rischi riguardanti la protezione dei dati
 - Rischi riguardanti la giurisdizione
 -

Rischi Organizzativi

■ Perdita di controllo

- forse il rischio considerato più grave
- CSC cede il controllo a CSP su molti aspetti critici per la sicurezza
- se la SLA di CSP lascia un gap rispetto alle necessità, non c'è modo di chiuderlo
- outsourcing e subcontracting potrebbero mettere in gioco attori non fidati
- CSC non può verificare autonomamente la compliance di CSP

■ Lock-in

- molti aspetti proprietari rendono difficile la migrazione
- anche on premise, ma almeno i dati restano in mano al CSC...

■ Supply Chain Failure

- outsourcing crea catene di fornitura dei servizi: forti quanto l'anello più debole

■ Interferenze tra politiche di sicurezza CSC-CSP

- CSC potrebbe desiderare controlli in conflitto con l'ambiente CSP, quindi non implementabili
- insicurezze di un CSC potrebbero diventare vulnerabilità di tutta la piattaforma che lo ospita

Rischi Tecnici

■ Economic denial of service (EdoS)

- Servizi pay-per-use che scalano automaticamente → un attacco di sovraccarico, invece di degradare le prestazioni, aumenta i costi

■ Vulnerabilità della piattaforma

- Un attacco all'infrastruttura potrebbe consentire l'accesso a tutte le VM
- Molto improbabile, ma devastante
- Più plausibile: compromissione dell'interfaccia di gestione

■ Cedimento dell'isolamento

- Multi-tenancy teoricamente permette di installare una VM malevola accanto a quelle della vittima
 - Possibili attacchi cross-VM via side channel
 - Possibile forzatura di migrazioni saturando l'host
- Intercettazione dei dati in transito
 - non tanto quelli dell'utente (che deve cautelarsi cifrando end-to-end)
 - operazioni trasparenti del CSP: sincronizzazioni, migrazioni, ecc.
 - raramente CSP fornisce garanzie di sicurezza su questi aspetti

Rischi legali

■ Rischi riguardanti la protezione dei dati

- legislazioni differenti
- potrebbe essere illegale trasferire alcuni tipi di dati in alcuni paesi
- CSP potrebbe spostare i dati tra i propri datacenter senza dirlo a CSC

■ Rischi riguardanti la giurisdizione

- CSP potrebbe essere costretto ad azioni dagli organi giudiziari sulla base di leggi del paese della sede legale, del paese dove sono collocati i datacenter, ecc...
- CSC potrebbe subire interruzioni di servizio o sequestro di dati sulla base di motivi che non sussistono nel proprio paese